

Agent Activity

A live-feeling dashboard of AI agents working through a task pipeline, built as a take-home assessment. Six agents run as a directed graph, hand work to one another, occasionally fail and retry, and the run loops indefinitely. Mock data, no backend.

Stack: Vite 8 · Three.js r186 · Tailwind CSS v4 · vanilla JS, no framework, no state library.

This document walks through how it is built and why each decision was made. Every code block is extracted from the source files at build time, so nothing here can drift out of step with the project.

1. The brief, as a checklist

Taken literally from the brief. Everything below is demonstrably true in the running page.

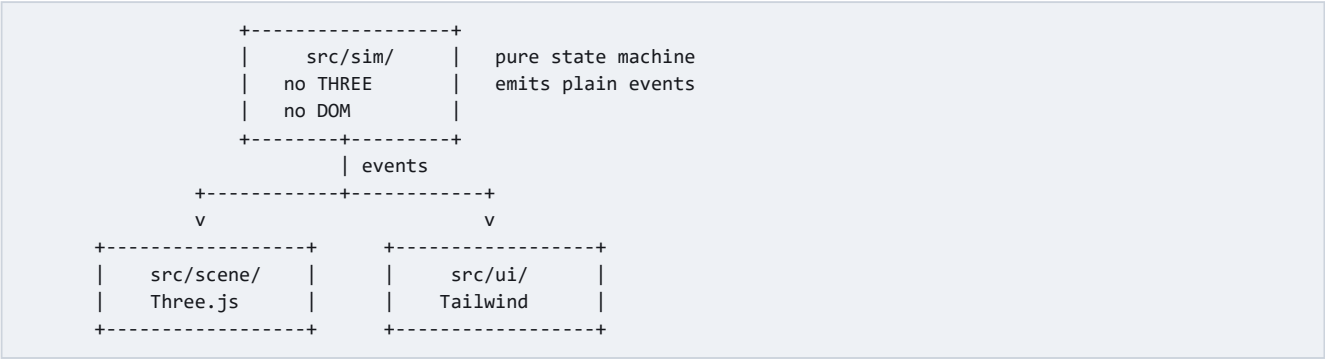
#	Requirement	Where it lives
R1	4–6 agent nodes in Three.js, laid out as a graph	scene/nodes.js, sim/layout.js
R2	pending → running → completed / failed, distinct visual per state	scene/nodes.js
R3	Edge animation on handoff — A completes, triggers B	scene/edges.js, sim/simulation.js
R4	HTML/Tailwind panel: live log with name, action, status, timestamp	ui/log.js, ui/roster.js
R5	Dark, dense, technical — infra dashboard, not a toy	style.css tokens, all of ui/
R6	Single page, mock/looping data, no backend	sim/simulation.js
R7	A few focused hours; judged on thinking and execution	scope held deliberately tight

R7 is treated as a real constraint, not a footnote. An over-built submission fails it, so the closing section lists what was left out on purpose.

2. The one decision that shapes everything

The simulation imports neither Three.js nor the DOM. It is a state machine that advances on a tick and emits plain, serialisable events. The 3D scene and the Tailwind panel are two independent subscribers to that stream. Neither view calls the other.

the seam



Three things fall out of that boundary, and they are why it earned an extra file or two on a build this size:

- The log panel and the 3D scene **cannot disagree** about an agent's state. They render the same events, not two copies of the truth.

- “Mock data, no backend” becomes an implementation detail of **one folder** rather than something smeared through the render loop. Pointing this at a real orchestrator means replacing `src/sim/` and touching nothing else.
- The simulation is runnable headless. It was developed and debugged in Node, with no browser, before any rendering existed at all.

Layer 1 — the simulation

Pure logic. No import in this folder reaches Three.js or document.

3. The pipeline, as data

Six agents — the top of the 4–6 range the brief allows, because six is what it takes to get both a fan-out and a fan-in:

the graph

```
ingest --> parse --+--> enrich ----+
                  |               +--> index --> report
                  +--> validate --+
```

A diamond rather than a chain. That choice does real work: it forces the layout to be computed instead of hand-placed, it exercises the handoff animation on six edges, and it gives `index` a genuine join condition — it may only start once *both* upstreams have delivered.

`src/sim/pipeline.js` — agent specs (first three shown)

```
export const AGENTS = [
  {
    id: 'ingest',
    name: 'ingest-agent',
    action: 'Fetching source documents',
    duration: [1400, 2200],
```

`src/sim/pipeline.js`

```
export const EDGES = [
  { from: 'ingest', to: 'parse' },
  { from: 'parse', to: 'enrich' },
  { from: 'parse', to: 'validate' },
  { from: 'enrich', to: 'index' },
  { from: 'validate', to: 'index' },
  { from: 'index', to: 'report' },
]
```

`src/sim/pipeline.js`

```
export const edgeId = (from, to) => `${from}-->${to}`

export function upstreamOf(agentId) {
  return EDGES.filter((edge) => edge.to === agentId).map((edge) => edge.from)
}

export function downstreamOf(agentId) {
  return EDGES.filter((edge) => edge.from === agentId).map((edge) => edge.to)
}
```

Note the failure rates sit on the two agents that call out to somebody else's system. The flakiest step in a real pipeline is always the one crossing a network boundary, so that is where the failure path belongs.

4. Layout, derived from the graph

Node positions are computed by longest-path layering rather than hand-placed, so editing EDGES re-arranges the diagram correctly. It is pure arithmetic — the scene consumes plain {x, y}.

`src/sim/layout.js`

```
export function computeLayout(orientation = 'horizontal') {
  const layers = assignLayers()
  const byLayer = new Map()

  for (const agent of AGENTS) {
    const layer = layers.get(agent.id)
    if (!byLayer.has(layer)) byLayer.set(layer, [])
    byLayer.get(layer).push(agent.id)
  }

  const lastLayer = Math.max(...layers.values())
  const { major, minor } = SPACING[orientation]
  const positions = new Map()

  for (const [layer, ids] of byLayer) {
    ids.forEach((id, row) => {
      const along = (layer - lastLayer / 2) * major
      // Centre the layer: a lone node sits on the axis, a pair straddles it.
      const across = (row - (ids.length - 1) / 2) * minor

      positions.set(
        id,
        orientation === 'horizontal'
          ? { x: along, y: -across }
          : { x: across, y: -along },
      )
    })
  }

  return positions
}
```

`src/sim/layout.js`

```
/** Longest path from a root — guarantees every edge points forward a layer. */
function assignLayers() {
  const layers = new Map(AGENTS.map((agent) => [agent.id, 0]))

  // The graph is small and acyclic; relaxing |V| times is plenty and avoids
  // hauling in a topological sort for six nodes.
  for (let pass = 0; pass < AGENTS.length; pass += 1) {
    for (const { from, to } of EDGES) {
      const candidate = layers.get(from) + 1
      if (candidate > layers.get(to)) layers.set(to, candidate)
    }
  }

  return layers
}
```

5. The event emitter

Six event types and three subscribers do not need a library.

`src/sim/emitter.js`

```
export function createEmitter() {
  const listeners = new Map()

  return {
    on(type, handler) {
      if (!listeners.has(type)) listeners.set(type, new Set())
      listeners.get(type).add(handler)
    }
  }
}
```

```

    return () => listeners.get(type).delete(handler)
  },

  emit(type, payload) {
    const handlers = listeners.get(type)
    if (!handlers) return
    // Copy before iterating: a handler may unsubscribe during dispatch.
    for (const handler of [...handlers]) handler(payload)
  },

  clear() {
    listeners.clear()
  },
}
}

```

The copy before iterating is not decoration: a handler may unsubscribe during dispatch, and mutating a Set mid-iteration would skip a listener.

6. The scheduler

This is the core of the submission. Two properties matter more than anything else in it.

Time is injected, not read

The caller supplies the delta — `sim.tick(deltaMs)` — rather than the simulation reading a clock. Pausing is then simply “stop calling tick”, and the log timestamps freeze together with the animation instead of racing ahead of it while paused.

`src/sim/simulation.js`

```

const MAX_ATTEMPTS = 2
const RETRY_DELAY_MS = 800
const HANDOFF_MS = 750
const RUN_RESTART_DELAY_MS = 3000

export function createSimulation() {

```

`src/sim/simulation.js` — timestamps derive from the injected clock

```

function timestamp() {
  return wallBase + clock
}

```

Because `wallBase` is captured once per run and everything else is accumulated tick time, a paused dashboard shows a paused clock. A wall-clock read would have produced timestamps that kept moving while nothing else did.

Starting, finishing, retrying

`src/sim/simulation.js`

```

function startAgent(agentId) {
  const agent = agents.get(agentId)
  const [min, max] = agent.spec.duration
  agent.duration = min + random() * (max - min)
  agent.elapsed = 0
  agent.attempt += 1
  setAgentState(agentId, 'running', { durationMs: agent.duration })
}

```

`src/sim/simulation.js`

```

function finishAgent(agentId) {
  const agent = agents.get(agentId)
  const base = agent.spec.failureRate ?? 0
  // A retry is far more likely to succeed than the first attempt — transient
  // faults are transient. Without this the loop stalls on the same node.

```

```

const rate = agent.attempt > 1 ? base * 0.15 : base

if (random() < rate) {
  const reason = FAILURE_REASONS[Math.floor(random() * FAILURE_REASONS.length)]
  const willRetry = agent.attempt < MAX_ATTEMPTS
  setAgentState(agentId, 'failed', { reason, willRetry })
  if (willRetry) agent.retryIn = RETRY_DELAY_MS
  return
}

setAgentState(agentId, 'completed', { durationMs: agent.duration })

for (const to of downstreamOf(agentId)) {
  const id = edgeId(agentId, to)
  handoffs.set(id, { id, from: agentId, to, elapsed: 0, duration: HANDOFF_MS })
  emitter.emit('handoff:start', {
    id,
    from: agentId,
    to,
    durationMs: HANDOFF_MS,
    at: timestamp(),
  })
}
}

```

The join condition

An agent starts only once every incoming edge has delivered its pulse. This is what makes index genuinely wait on both enrich and validate.

`src/sim/simulation.js`

```

function tryStart(agentId) {
  const agent = agents.get(agentId)
  if (agent.state !== 'pending' || agent.retryIn > 0) return

  const ready = upstreamOf(agentId).every((from) =>
    agent.delivered.has(edgeId(from, agentId)),
  )
  if (ready) startAgent(agentId)
}

```

The tick

`src/sim/simulation.js`

```

function tick(deltaMs) {
  if (restartCountdown !== null) {
    restartCountdown -= deltaMs
    if (restartCountdown <= 0) startRun()
    return
  }

  clock += deltaMs

  for (const agent of agents.values()) {
    if (agent.retryIn > 0) {
      agent.retryIn -= deltaMs
      if (agent.retryIn <= 0) {
        agent.retryIn = 0
        setAgentState(agent.spec.id, 'pending', { retrying: true })
        startAgent(agent.spec.id)
      }
      continue
    }

    if (agent.state !== 'running') continue
    agent.elapsed += deltaMs
    if (agent.elapsed >= agent.duration) finishAgent(agent.spec.id)
  }
}

```

```

for (const handoff of [...handoffs.values()]) {
  handoff.elapsed += deltaMs
  if (handoff.elapsed < handoff.duration) continue

  handoffs.delete(handoff.id)
  agents.get(handoff.to).delivered.add(handoff.id)
  emitter.emit('handoff:end', {
    id: handoff.id,
    from: handoff.from,
    to: handoff.to,
    at: timestamp(),
  })
  tryStart(handoff.to)
}

if (isSettled()) endRun()

```

Ending a run, and the blocked case

A run is over when nothing can advance it: no agent running, nothing in transit, no retry queued. Agents still pending at that moment are blocked behind a permanent failure and will never run — the panel says so rather than leaving them looking queued.

`src/sim/simulation.js`

```

function isSettled() {
  if (handoffs.size > 0) return false
  for (const agent of agents.values()) {
    if (agent.state === 'running' || agent.retryIn > 0) return false
  }
}

```

`src/sim/simulation.js`

```

function endRun() {
  const status = agents.get('report').state === 'completed' ? 'completed' : 'failed'

  // Agents still pending at this point are blocked behind a failure and will
  // never run. Say so explicitly rather than leaving them looking queued.
  const blocked = [...agents.values()]
    .filter((agent) => agent.state === 'pending')
    .map((agent) => agent.spec.id)

  emitter.emit('run:end', {
    runId,
    status,
    blocked,
    at: timestamp(),
    durationMs: clock,
  })

  restartCountdown = RUN_RESTART_DELAY_MS
}

```

Deterministic randomness

A seeded PRNG rather than `Math.random`: runs still vary, but a seed reproduces one exactly when something needs debugging.

`src/sim/simulation.js`

```

/** Small seeded PRNG: runs vary, but a seed reproduces one exactly when debugging. */
function mulberry32(seed) {
  let a = seed
  return function next() {
    a |= 0
    a = (a + 0x6d2b79f5) | 0
    let t = Math.imul(a ^ (a >> 15), 1 | a)
    t = (t + Math.imul(t ^ (t >> 7), 61 | t)) ^ t
    return ((t ^ (t >> 14)) >> 0) / 4294967296
  }
}

```

7. Verifying it without a browser

Because the simulation is pure, it runs in Node. It was driven through 400,000 ticks before any rendering existed, which is how the rare permanent-failure path was confirmed to terminate cleanly instead of deadlocking the loop:

headless run trace (Node, no browser)

```
run 1 -> completed in 17.4s blocked=[]
  ingest    running   attempt=1
  ingest    completed attempt=1
  parse     running   attempt=1
  parse     completed attempt=1
  enrich    running   attempt=1
  validate  running   attempt=1      <- fan-out: two agents in parallel
  validate  completed attempt=1
  enrich    failed    attempt=1 :: rate limit exceeded (429)
  enrich    pending   attempt=1      <- retry scheduled
  enrich    running   attempt=2
  enrich    completed attempt=2
  index     running   attempt=1      <- join satisfied: both upstreams in
  report    completed attempt=1

FAILED RUN 2:  blocked=[index,report]    <- permanent failure, run still ends
runs=360 failedRuns=10 (2.8%)
```

That 2.8% is deliberate. A retry is far more likely to succeed than a first attempt — transient faults are transient — so the pipeline usually recovers, and a genuinely blocked run stays a rare event rather than a constant annoyance.

Layer 2 — the HTML panel

8. One palette, two consumers

The state colours are declared once as Tailwind v4 @theme tokens and read back out in JavaScript for the Three.js materials. A state can therefore never be one colour in the panel and a different one in the scene.

src/style.css — the tokens

```
@theme {
  --color-surface-void: #08090c;
  --color-surface-panel: #0c0e13;
  --color-surface-raised: #11141b;
  --color-surface-line: #1e222c;
  /* Backdrop dots and edge lines sit a step lighter than the panel chrome, so
     the canvas reads as a workspace rather than an empty void. */
  --color-surface-grid: #39435a;
  --color-surface-edge: #414d63;
  --color-surface-port: #3f4a5e;

  --color-ink-bright: #e4e7ee;
  --color-ink-base: #949cad;
  --color-ink-dim: #5b6373;
  --color-ink-faint: #39404e;

  --color-state-pending: #5b6373;
  --color-state-running: #f0a02c;
  --color-state-completed: #35c88a;
  --color-state-failed: #e8544a;

  --font-mono: "JetBrains Mono", ui-monospace, "SF Mono", Menlo, Consolas, monospace;
```

src/tokens.js – reading them back

```
const css = getComputedStyle(document.documentElement)

function read(name) {
  const value = css.getPropertyValue(name).trim()
  if (!value) throw new Error(`Missing CSS custom property: ${name}`)
  return value
}

export const STATE_COLORS = {
  pending: read('--color-state-pending'),
  running: read('--color-state-running'),
  completed: read('--color-state-completed'),
  failed: read('--color-state-failed'),
}

export const SURFACE_COLORS = {
```

Why this matters. The obvious alternative is a JS colour object that the CSS duplicates. That works right up until someone edits one of the two, and then the dashboard quietly lies: a node glowing amber next to a log line rendered in a slightly different amber. Reading the computed CSS makes the duplication impossible.

9. The live log

The simulation never emits a “log event”. It emits domain events, and the log derives readable lines from them here — formatting is presentation, so it lives with the other presentation code.

src/ui/log.js

```
agentState(event) {
  // The initial pending sweep at run start is bookkeeping, not activity.
  if (event.quiet) return

  const message =
    event.state === 'failed'
      ? `${event.reason}${event.willRetry ? ' · scheduling retry' : ' · giving up'}`
      : event.state === 'completed'
        ? `${event.action} · ${(event.durationMs / 1000).toFixed(2)}s`
        : event.state === 'running' && event.attempt > 1
          ? `${event.action} · attempt ${event.attempt}`
          : event.action

  line({
    at: event.at,
    name: event.name,
    status: STATUS_LABEL[event.state],
    statusClass: STATE_TEXT[event.state],
    message,
  })
},
```

src/ui/log.js – terminal-style autoscroll

```
function atBottom() {
  return root.scrollHeight - root.scrollTop - root.clientHeight < 32
}
```

Autoscroll only sticks while the reader is already at the bottom, exactly as a terminal behaves. Scroll up to read something and the feed stops yanking you back down.

10. The roster

Rows are built once and then mutated. Re-rendering the list on every event would reset scroll position and make the running timer flicker.

src/ui/roster.js

```
function setState(event) {
  const row = rows.get(event.agentId)
  row.state = event.state
  row.dot.className = `mt-1.5 size-1.5 shrink-0 rounded-full ${STATE_BG[event.state]} ${
    event.state === 'running' ? 'animate-state-pulse' : ''
  }`
}
```

11. Labels are projected HTML, not 3D text

A dashboard lives or dies on legible type. Projected DOM gives crisp subpixel rendering, real font features and the same Tailwind tokens as the side panel — none of which a texture atlas or extruded geometry would.

src/ui/labels.js

```
/** Called per frame with the projected pixel position of each node. */
place(agentId, x, y) {
  // transform rather than left/top keeps the label on the compositor and off
  // the layout path, which matters when it moves every frame. The -50% is
  // folded in here because an inline transform would override a Tailwind
  // `-translate-x-1/2` class outright.
  labels.get(agentId).el.style.transform =
    `translate(calc(${x}px - 50%), ${y}px)`
},
}
```

src/scene/viewport.js — the projection

```
/** Container-relative pixel position of a world point, for HTML labels. */
project(position, out) {
  const ndc = position.clone().project(camera)
  out.x = (ndc.x * 0.5 + 0.5) * container.clientWidth
  out.y = (-ndc.y * 0.5 + 0.5) * container.clientHeight
  return out
},
```

Layer 3 — the Three.js scene

12. The viewport

A fixed camera, framed by solving the frustum

This is a dashboard, so the composition is part of the design; an orbit control would only invite a viewer to drag the thing they came to read out of alignment. The camera distance is solved from the frustum rather than tuned by hand, so a narrow window or a different graph cannot crop a node off the edge.

src/scene/viewport.js

```
function fitCamera() {
  const width = container.clientWidth
  const height = container.clientHeight
  const halfFov = (camera.fov * Math.PI) / 360

  // The labels are HTML at a fixed pixel size and do not scale with the
  // camera, so the room they need is reserved in pixels rather than in world
  // units. Reserving it in world units is what let the bottom label fall off
  // a short screen: the same world padding buys fewer and fewer pixels the
  // smaller the viewport gets, which is exactly backwards.
  const usableW = Math.max(width - framing.reserveX, width * 0.45)
  const usableH = Math.max(height - framing.reserveY, height * 0.45)

  const forHeight = ((framing.height * height) / usableH) / 2 / Math.tan(halfFov)
```

```

const forWidth =
  ((framing.width * width) / usableW) / 2 / (Math.tan(halfFov) * camera.aspect)

camera.position.z = Math.max(forHeight, forWidth) * 1.02
camera.lookAt(target)
camera.updateProjectionMatrix()
}

```

src/main.js – the box it must keep in shot, derived from the layout

```

/** World-space box the camera must keep in shot, derived from the layout. */
function framingFor(positions, flow) {
  const xs = [...positions.values()].map((p) => p.x)
  const ys = [...positions.values()].map((p) => p.y)
  const vertical = flow === 'vertical'

  return {
    // The graph's own extent, cards included, in world units.
    width: Math.max(...xs) - Math.min(...xs) + CARD.width,
    height: Math.max(...ys) - Math.min(...ys) + CARD.height,

    // Pixel room for the label hanging under each card, plus surrounding air.
    // Stacked, width is the scarce axis and there is height to give away; side
    // by side it is the other way round. The vertical figure is the smallest
    // that never clips the bottom label on a 320x568 screen – anything larger
    // is scale taken off the cards for nothing.
    reserveX: vertical ? 44 : 170,
    reserveY: vertical ? 72 : 88,
  }
}

```

One render loop

Everything animated subscribes to a single loop. Nothing else in the project calls `requestAnimationFrame`.

src/scene/viewport.js – the clock

```

// Timer rather than the (now deprecated) Clock. `connect` opts into the Page
// Visibility API: while the tab is hidden the delta is held at zero and the
// timer resets on return, so a backgrounded dashboard resumes where it left
// off instead of fast-forwarding the whole pipeline the moment you look back.
const timer = new Timer()
timer.connect(document)

```

src/scene/viewport.js – the loop

```

/** Advance every subscriber by `delta` seconds and draw one frame. */
function step(delta) {
  for (const update of updaters) update(delta, timer.getElapsed())
  composer.render()
}

renderer.setAnimationLoop((timestamp) => {
  timer.update(timestamp)
  step(timer.getDelta())
})

```

A detail worth knowing. `THREE.Clock` is deprecated in r186, and `Timer` now lives in *three core*, not *three/addons/misc/* — importing it from *addons 404s* and blanks the page. `connect(document)` opts into the Page Visibility API, so a backgrounded dashboard resumes where it left off instead of fast-forwarding the whole pipeline the moment you look back at it.

Bloom, and why there is no tone mapping

src/scene/viewport.js

```
// Bloom is what makes an emissive node read as "lit from within" rather than
// "a brightly coloured shape". Kept restrained: the threshold sits above the
// dark surfaces and the grid so only lit state colour blooms, and the
// strength stops well short of blowing the node out to white – a glowing
// blob loses the hexagon silhouette that identifies it as a node.
//
// No tone mapping, deliberately. ACES would roll the highlights off nicely
// but it also shifts every colour, and these colours are the same tokens the
// HTML panel uses. The two views agreeing matters more than the filmic curve.
const bloom = new UnrealBloomPass(new Vector2(1, 1), 0.55, 0.4, 0.55)
composer.addPass(bloom)
```

13. The agent cards

The first pass drew hexagonal prisms. Against a real orchestration canvas they read as abstract solids — decorative rather than diagrammatic. A rounded card carrying a glyph is the vocabulary the domain already uses, so the diagram explains itself before anyone reads a label. The card is flat rather than extruded — section 18 explains what that fixed.

src/scene/nodes.js

```
export const CARD = { width: 1.62, height: 1.24, radius: 0.3 }

/**
 * The card is flat, not an extruded box.
 *
 * An extruded card has side walls, and under perspective every card off the
 * centre axis shows the wall on its inward-facing side. Worse, the border is
 * drawn on the front face, so it projects inset from the card's silhouette –
 * which reads as a second border running through the body of the node rather
 * than as depth. Flattening the card makes the border and the silhouette the
 * same edge at every position on screen.
 *
 * Depth in this scene comes from the layout, the camera and the backdrop, none
 * of which can misalign an outline.
 */
const LAYER = 0.004
```

src/scene/nodes.js

```
function roundedRectShape() {
  const { width: w, height: h, radius: r } = CARD
  const x = w / 2
  const y = h / 2

  const shape = new Shape()
  shape.moveTo(-x + r, -y)
  shape.lineTo(x - r, -y)
  shape.quadraticCurveTo(x, -y, x, -y + r)
  shape.lineTo(x, y - r)
  shape.quadraticCurveTo(x, y, x - r, y)
  shape.lineTo(-x + r, y)
  shape.quadraticCurveTo(-x, y, -x, y - r)
  shape.lineTo(-x, -y + r)
  shape.quadraticCurveTo(-x, -y, -x + r, -y)
  return shape
}
```

src/scene/nodes.js

```
/** Flat rounded rectangle. See the note on CARD for why it is not extruded. */
function buildCardGeometry() {
  return new ShapeGeometry(roundedRectShape(), 12)
}
```

State treatment

The card surface stays dark and neutral in every state; the border carries the state. A colour-washed card would fight the icon sitting on top of it.

src/scene/nodes.js

```
setState(next) {
  state = next
  sinceChange = 0

  const color = new Color(STATE_COLORS[next])
  surfaceMaterial.emissive.copy(color)
  borderMaterial.color.copy(color)
  beamMaterial.color.copy(color)

  // The glyph lifts out of the card as soon as the agent is doing anything,
  // and dims back down once it is finished, so the eye lands on live work.
  iconMaterial.opacity = next === 'pending' ? 0.62 : next === 'running' ? 1 : 0.85

  beamMaterial.opacity = 0
  beam.visible = next === 'running'
  if (next === 'running') beamOffset = 0
},
```

The running indicator: a travelling beam

The original progress ring was precise but dated, and it duplicated information the panel already showed twice — the per-agent live timer and the run progress bar. A bright segment running around the card border is the current idiom for indeterminate work, and it costs nothing: it is a `setDrawRange` window sliding along an outline that already exists.

src/scene/nodes.js

```
const OUTLINE_POINTS = 168
const BEAM_POINTS = 30
const BEAM_PERIOD = 1.15 // seconds for one lap

const cardGeometry = buildCardGeometry()
const outlinePath = buildOutlinePoints()
const borderGeometry = toLineGeometry(outlinePath)
// The beam window slides along a doubled copy of the loop, so it can cross the
// seam without the draw range having to wrap and visibly stutter.
const beamGeometry = toLineGeometry([...outlinePath, ...outlinePath])
const iconGeometry = new PlaneGeometry(0.56, 0.56)
const portGeometry = new CircleGeometry(0.055, 16)
```

src/scene/nodes.js – the beam (running branch)

```
update(delta, elapsed, reducedMotion) {
  sinceChange += delta

  if (state === 'running') {
    surfaceMaterial.emissiveIntensity = reducedMotion
      ? 0.03
      : 0.028 + Math.sin(elapsed * 4.2) * 0.016
    // The resting outline drops right back while the beam is running, so
    // the bright segment reads as one travelling light rather than as a
    // slightly brighter patch of an already-lit border.
    borderMaterial.opacity = 0.16

    if (reducedMotion) {
      // With motion suppressed the beam would be a flicker, so show the
      // whole outline lit instead: still unmistakably "this one is live".
      beamGeometry.setDrawRange(0, OUTLINE_POINTS)
      beamMaterial.opacity = 0.95
      return
    }

    beamOffset = (beamOffset + (delta / BEAM_PERIOD) * OUTLINE_POINTS) % OUTLINE_POINTS
```

```

    beamGeometry.setDrawRange(Math.floor(beamOffset), BEAM_POINTS)
    beamMaterial.opacity = 0.95
    return
  }

```

The doubled outline. A draw-range window that runs off the end of the buffer has to wrap, and the wrap is visible as a stutter once per lap. Storing the loop twice end-to-end lets the window cross the seam as one continuous run of vertices. The cost is 168 extra vertices per node.

14. Icons

SVG path data on a 24×24 grid, rendered to a canvas with Path2D and used as a texture. No sprite sheet, no network request, and nothing async to sequence against the first frame — the texture is ready the moment the node is built.

[src/scene/icons.js](#)

```

export function createIconTexture(agentId) {
  const icon = ICONS[agentId]
  const canvas = document.createElement('canvas')
  canvas.width = SIZE
  canvas.height = SIZE

  const ctx = canvas.getContext('2d')
  const scale = SIZE / 24
  ctx.scale(scale, scale)

  ctx.strokeStyle = icon.color
  ctx.lineWidth = 1.6
  ctx.lineCap = 'round'
  ctx.lineJoin = 'round'

  for (const path of icon.paths) ctx.stroke(new Path2D(path))

  const texture = new CanvasTexture(canvas)
  texture.colorSpace = SRGBColorSpace
  // The plane is small on screen and never magnified much; linear filtering
  // without mipmaps keeps the strokes crisp rather than muddy.
  texture.minFilter = LinearFilter
  texture.generateMipmaps = false
  return texture
}

```

[src/scene/icons.js – one icon definition](#)

```

ingest: {
  color: '#2dd4bf',
  paths: ['M12 3.5v10', 'M7.5 9l4.5 4.5L16.5 9', 'M4 18.5h16'],
},

```

15. Edges and the handoff

Each edge leaves the source card's right port and arrives at the target's left port as a cubic bezier with horizontal tangents — the shape every workflow canvas uses, and the reason the fan-out from parse reads as two branches of one flow instead of two unrelated diagonals.

[src/scene/edges.js](#)

```

// Leave and arrive at the ports, not the card centres, so the line
// meets the node where the little circle is.
const start = vertical
  ? new Vector3(from.x, from.y - CARD.height / 2 - PORT_GAP, 0)
  : new Vector3(from.x + CARD.width / 2 + PORT_GAP, from.y, 0)
const end = vertical
  ? new Vector3(to.x, to.y + CARD.height / 2 + PORT_GAP, 0)
  : new Vector3(to.x - CARD.width / 2 - PORT_GAP, to.y, 0)

// Pull the control points straight out along the flow. The flatter the
// handles, the more the curve reads as "flows onward" than "wanders".

```

```

const span = vertical ? start.y - end.y : end.x - start.x
const reach = Math.max(span * 0.5, 0.5)

edge.curve = new CubicBezierCurve3(
  start,
  vertical
    ? new Vector3(start.x, start.y - reach, 0)
    : new Vector3(start.x + reach, start.y, 0),
  vertical
    ? new Vector3(end.x, end.y + reach, 0)
    : new Vector3(end.x - reach, end.y, 0),
  end,
)

```

src/scene/edges.js

```

update(sim, reducedMotion) {
  for (const [id, edge] of edges) {
    if (!edge.active) continue

    const progress = sim.handoffProgress(id)
    if (progress === null) continue

    if (reducedMotion) {
      // No travelling dot when motion is suppressed. The edge still lights
      // up, so the handoff is reported – just as a change of state rather
      // than as something moving across the screen.
      edge.pulse.visible = false
      edge.lineMaterial.color.set(STATE_COLORS.completed)
      edge.lineMaterial.opacity = 0.75
      continue
    }

    edge.curve.getPoint(progress, edge.pulse.position)

    // Fade in and out at the ends so the pulse appears to leave one card
    // and be absorbed by the next, rather than popping in mid-air.
    const ends = Math.min(progress, 1 - progress) / 0.15
    edge.pulseMaterial.opacity = Math.min(1, ends) * 0.95

    // The line lights up behind the pulse: the edge itself records which
    // way the work flowed, and stays lit for the rest of the run.
    edge.lineMaterial.color.set(STATE_COLORS.completed)
    edge.lineMaterial.opacity = 0.25 + progress * 0.5
  }
},

```

16. Why the pulse *is* the state change

When an agent completes, a pulse travels the edge to each downstream agent — and that agent flips to running **when the pulse lands**, not when the upstream completed.

This is the single most deliberate choice in the visual layer. The animation is not decorating a state change that already happened; it *is* that change's timeline. The simulation owns transit as a timed phase and emits its duration; the scene draws inside that window; the downstream agent starts when the window closes. Cause and effect therefore read correctly on screen, which is the whole point of the requirement.

src/sim/simulation.js – arrival starts the downstream agent

```

handoffs.delete(handoff.id)
agents.get(handoff.to).delivered.add(handoff.id)
emitter.emit('handoff:end', {
  id: handoff.id,
  from: handoff.from,
  to: handoff.to,
  at: timestamp(),
})
tryStart(handoff.to)

```

Putting it together

17. Responsive: one graph, two orientations

The pipeline is five layers long and two wide, so laid out left to right it is a roughly 5:1 letterbox. On a portrait phone the camera would have to pull back until the cards were unreadable, so the graph re-flows top to bottom instead, putting its long axis on the screen's long axis.

It is the same graph either way. The layout already mapped (layer, row) to (x, y); the orientation only decides which of the two the layer feeds. The pipeline data, the scheduler and the events are untouched.

`src/sim/layout.js`

```
for (const [layer, ids] of byLayer) {
  ids.forEach((id, row) => {
    const along = (layer - lastLayer / 2) * major
    // Centre the layer: a lone node sits on the axis, a pair straddles it.
    const across = (row - (ids.length - 1) / 2) * minor

    positions.set(
      id,
      orientation === 'horizontal'
        ? { x: along, y: -across }
        : { x: across, y: -along },
    )
  })
}
```

Driven by the measured stage, not a CSS breakpoint

The ResizeObserver that already sized the canvas now also reports its shape. A `md` breakpoint would have missed a narrow desktop window and a split screen; the box the graph is actually drawn into cannot.

`src/main.js`

```
function orientationFor(stage) {
  return stage.clientWidth / Math.max(stage.clientHeight, 1) < 1.15
    ? 'vertical'
    : 'horizontal'
}
```

`src/main.js`

```
viewport.onResize(() => {
  const next = orientationFor(el('stage'))
  if (next === orientation) return
  orientation = next

  const layout = computeLayout(next)
  nodes.setLayout(layout, next)
  edges.setLayout((agentId) => nodes.positionOf(agentId), next)
  labels.setCompact(next === 'vertical')
  viewport.setFraming(framingFor(layout, next))
})
```

Nothing is rebuilt on the switch

Cards are repositioned and edge curves rewritten in place. State lives in each node's closure rather than in its position, so a run in flight carries straight on across the change with no snapshot to replay — the same separation the whole project rests on, paying out somewhere it was not designed for.

Label space had to move from world units to pixels

The labels are HTML at a fixed size and do not scale with the camera, so world-unit padding buys fewer pixels the smaller the screen gets — exactly backwards. The bottom label fell off a 375×667 screen. The fit now takes a reserve in pixels and solves for the distance that leaves it free.

`src/scene/viewport.js`

```
function fitCamera() {
  const width = container.clientWidth
  const height = container.clientHeight
  const halfFov = (camera.fov * Math.PI) / 360

  // The labels are HTML at a fixed pixel size and do not scale with the
  // camera, so the room they need is reserved in pixels rather than in world
  // units. Reserving it in world units is what let the bottom label fall off
  // a short screen: the same world padding buys fewer and fewer pixels the
  // smaller the viewport gets, which is exactly backwards.
  const usableW = Math.max(width - framing.reserveX, width * 0.45)
  const usableH = Math.max(height - framing.reserveY, height * 0.45)

  const forHeight = ((framing.height * height) / usableH) / 2 / Math.tan(halfFov)
  const forWidth =
    ((framing.width * width) / usableW) / 2 / (Math.tan(halfFov) * camera.aspect)

  camera.position.z = Math.max(forHeight, forWidth) * 1.02
  camera.lookAt(target)
  camera.updateProjectionMatrix()
}
```

Stacked, the labels also drop the -agent suffix and go *up* a point. Counter-intuitive but correct: every node here is an agent, so the suffix carries no information in the diagram, and dropping it frees far more width than the extra point costs.

One known trade-off: edges cross the stacked labels

Flowing top to bottom, an edge leaves a card's lower port at the card's centre x and runs straight down — and the label is centred under the card at that same x, so the line passes through the middle of the text. Projecting every edge and every label box with the app's own camera shows the shape of it exactly:

`edge / label crossing check`

vertical	414x502	ingest->parse crosses "ingest"; parse->enrich crosses "parse"; parse->validate crosses "parse"; enrich->index crosses "enrich"; validate->index crosses "validate"; index->report crosses "index"
horizontal	1176x676	no edge crosses any label

The labels are HTML over the canvas, so the letters are painted on top of the line — the issue is legibility, not stacking order. A backplate in the canvas colour was tried and rejected on review: it read as a black box behind the text rather than as part of the diagram, and it cost scale, since a larger label forces the camera further back.

The option not taken was to move the label rather than hide the line — beside the card when stacked, where there is spare width and no edge runs. That costs the diagram a consistent label position between the two orientations, which is why the crossing is accepted instead.

Verified without a browser, again

The development tab was backgrounded, so `requestAnimationFrame` never fired and the labels were never placed — nothing in the DOM to measure. But the projection is arithmetic and three's camera classes run headless, so the on-screen geometry was reproduced in Node (real label widths measured once in the browser, everything else computed as the app computes it) and asserted across six device sizes for overlap and clipping. That is what caught the clipped label, and a sweep then set the reserve at the smallest value that never clips.

headless geometry check

320x568	card	31x24px	rowgap	31px	overlaps	none	clipped	none
375x667	card	38x30px	rowgap	38px	overlaps	none	clipped	none
393x851	card	51x40px	rowgap	51px	overlaps	none	clipped	none
414x896	card	55x42px	rowgap	54px	overlaps	none	clipped	none
768x1024	card	64x49px	rowgap	63px	overlaps	none	clipped	none
1176x676	card	116x87px			overlaps	none	clipped	none

18. The composition root

src/main.js is the only module that knows all three layers exist.

src/main.js

```
sim.on('run:start', (event) => {
  runBar.start(event.runId)
  edges.reset()
  log.runBoundary(`run #${event.runId.toString(16).padStart(4, '0')} started`, event.startedAt)
})

sim.on('agent:state', (event) => {
  nodes.setState(event.agentId, event.state)
  labels.setState(event.agentId, event.state)
  roster.setState(event)
  log.agentState(event)
  runBar.setProgress(roster.progress())
})

sim.on('handoff:start', (event) => {
  edges.beginHandoff(event.id)
  log.handoff(event, agentNames)
})

sim.on('handoff:end', (event) => {
  edges.endHandoff(event.id)
})

sim.on('run:end', (event) => {
```

src/main.js

```
viewport.onFrame((delta, elapsed) => {
  if (!paused) sim.tick(delta * 1000)

  nodes.update(delta, elapsed, viewport.reducedMotion)
  edges.update(sim, viewport.reducedMotion)
  roster.tick(sim)
  runBar.tick(sim.elapsed)

  for (const agent of AGENTS) {
    labelAnchor.copy(nodes.positionOf(agent.id))
    labelAnchor.y -= CARD.height / 2 + 0.3
    viewport.project(labelAnchor, projected)
    labels.place(agent.id, projected.x, projected.y)
  }
})
```

19. Bugs found by looking, not by reasoning

The first three were caught by rendering the page and inspecting it, not by reading the code — the third from a screenshot sent back by the reviewer. Each is commented at the site of the fix.

The canvas was 25% too large

`renderer.setSize(w, h, false)` skips setting the canvas CSS size, so the canvas kept its intrinsic pixel size. On a display with `devicePixelRatio` 1.25 the scene rendered oversized and clipped, and the projected HTML labels drifted from their nodes by exactly that ratio — which is how it was diagnosed: every label sat at 0.8× its node's x position.

`src/scene/viewport.js`

```
- renderer.setSize(width, height, false)
+ renderer.setSize(width, height)
```

The icons were buried inside the card

`ExtrudeGeometry`'s `bevel` pushes the front face a further `bevelThickness` beyond `depth / 2`. Everything placed at `depth / 2` — icons, borders, ports — rendered inside the plate and was invisible. The flattening described next removed the extrusion altogether, so this class of mistake can no longer occur.

A second border ran through the body of the off-axis cards

Reported from a screenshot: a stray border appeared inside `parser-agent` on its right, and inside `index-agent` and `report-agent` on their left. The pattern *is* the diagnosis — those are exactly the cards off the centre axis, and the affected side was always the one facing the centre. The two cards sitting on the axis were clean.

- An extruded card has side walls, and under perspective a card left of centre shows its right wall while a card right of centre shows its left one.
- Worse, the border is a line drawn on the *front face*. For an off-axis card that face projects inset from the card's silhouette, so the outline stopped tracing the shape on screen — roughly a five-pixel gap on the outermost card.

The fix was to stop extruding. The border and the silhouette are now the same edge at every position on screen and at any window size, with no per-card correction. What was given up is a bevel highlight barely visible at this scale; depth here comes from the layout, the camera and the backdrop, none of which can misalign an outline.

`src/scene/nodes.js`

```
- new ExtrudeGeometry(roundedRectShape(),
-   { depth: CARD.depth, bevelEnabled: true, bevelThickness: BEVEL, ... })
+ new ShapeGeometry(roundedRectShape(), 12)
```

A shared geometry gave every node the same progress

In the first pass the progress arc was one shared `BufferGeometry`. `setDrawRange` is a property of the geometry, not of the mesh — so all six nodes displayed whichever progress was written last. Caught while writing the code, but it is the same class of mistake: shared state hiding behind an object that looks per-instance.

20. Accessibility and performance

- **prefers-reduced-motion** is respected across both views: the beam stops travelling and lights the whole outline instead, edge pulses stop moving while the edge still lights up, and the CSS pulse on the panel dots is disabled. The dashboard still reads correctly from discrete state changes alone.
- **Pausing freezes everything together** — scene, timers, log timestamps and the CSS dot animation — because a paused stream that still looks alive is a lie.
- **Pixel ratio is capped at 2**. Beyond that the extra fragments cost far more than they show.
- **Per-frame work is allocation-free** where it matters: the beam moves a draw range, the pulse writes into an existing vector, and labels are positioned with `transform` so they stay on the compositor and off the layout path.
- **GPU resources are disposed** on hot reload, so dev sessions do not leak WebGL contexts.

21. Deliberately out of scope

Listed so the omissions read as decisions rather than gaps.

- No backend, websocket or persistence — the brief rules it out.
- No framework and no state library. Three subscribers do not need React.
- No camera controls. A considered fixed angle suits a dashboard; a draggable one invites the viewer to break the composition.
- No agent drill-down, filtering or timeline scrubber. All plausible next features; none asked for, and each would cost more than it shows.
- No test suite. The simulation being headless-testable is the argument for the seam, but writing the tests is not what this brief is grading.

The production bundle is roughly 600 kB, almost entirely Three.js. Tree-shaking already applies; dynamic imports would cut the initial load further if that mattered here.

22. Running it

commands

```
npm install
npm run dev      # dev server with HMR
npm run build    # production build to dist/
npm run preview  # serve the production build
```